

```
// Glue between the HTML page and the  
rest of JARVIS. Keep this file thin.
```

```
import { PROVIDER_PRESETS,  
loadSettings, saveSettings,  
getProviderSettings, speechLang }  
from "./config.js";  
import { createProvider } from  
"./providers.js";  
import { Brain } from "./brain.js";  
import { Listener, Speaker } from  
"./voice.js";  
import { Controller } from  
"./controller.js";  
import { friendlyMessage, JarvisError  
} from "./errors.js";
```

```
const $ = (id) =>  
document.getElementById(id);  
const settings = loadSettings();  
const brain = new  
Brain(createProvider(getProviderSetti  
ngs(settings)));  
const listener = new Listener({ lang:  
speechLang(settings) });  
const speaker = new Speaker({ lang:
```

```

speechLang(settings) });

const LABELS = {
  idle: "TAP TO SPEAK",
  listening: "LISTENING...\ntap when
done",
  thinking: "THINKING...\ntap to
cancel",
  speaking: "SPEAKING...\ntap to stop",
};

const ui = {
  setState(state, text, isError) {
    const talk = $("talk");
    talk.className = state;
    talk.textContent = LABELS[state];
    talk.style.whiteSpace = "pre-
line";
    const status = $("status");
    status.className = isError ?
"error" : "";
    status.textContent = text || ({
idle: "Ready.", listening:
"Listening...", thinking: "Thinking...",
speaking: "Speaking..." })[state];
  },
  showYou(text) {
$("you").textContent = text ||
"\u00a0"; },

```

```

    showJarvis(text) {
    $("jarvis").textContent = text ||
    "\u00a0"; },
    };

    const controller = new Controller({
    listener, speaker, getBrain: () =>
    brain, ui });

    $("talk").addEventListener("click",
    () => { controller.press(); });
    $("new-
    chat").addEventListener("click", ()
    => { brain.reset(); ui.showYou("");
    ui.showJarvis("");
    ui.setState("idle", "New
    conversation."); });

    // ----- settings screen -----
    function fillSettingsForm(name) {
        const preset =
    PROVIDER_PRESETS[name];
        const cfg =
    getProviderSettings(settings, name);
        $("provider").value = name;
        $("api-key").value = cfg.apiKey;
        $("model").value = cfg.model;
        $("base-url").value = cfg.baseUrl;
        $("lang").value = settings.lang;
    }

```

```
const link = $("key-link");
link.hidden = !preset.keyUrl;
link.href = preset.keyUrl || "#";
$("base-url-row").hidden = name !==
"custom";
}

function openSettings() {
  $("settings").hidden = false;
  $("settings-msg").textContent = "";

  fillSettingsForm(settings.provider);
  $("settings").scrollIntoView();
}

for (const [name, preset] of
Object.entries(PROVIDER_PRESETS)) {
  const opt =
document.createElement("option");
  opt.value = name;
  opt.textContent = preset.label;
  $("provider").appendChild(opt);
}
$("provider").addEventListener("change", (e) =>
fillSettingsForm(e.target.value));
$("open-
settings").addEventListener("click",
openSettings);
```

```

$("close-
settings").addEventListener("click",
() => { $("settings").hidden = true;
window.scrollTo(0, 0); });
$("save").addEventListener("click",
() => {
    const name = $("provider").value;
    settings.provider = name;
    settings.lang =
$("lang").value.trim();
    settings.providers[name] = {
        apiKey: $("api-
key").value.trim(),
        model: $("model").value.trim(),
        baseUrl: $("base-
url").value.trim(),
    };
    const saved =
saveSettings(settings);
    brain.provider =
createProvider(getProviderSettings(se
ttings));
    listener.lang = speaker.lang =
speechLang(settings);
    $("settings-msg").textContent =
saved ? "Saved. Close this and tap
the big button." : "Could not save on
this browser (private mode?).";
});

```

```
// ----- startup checks -----  
if (!Listener.isSupported())  
    ui.setState("idle",  
        friendlyMessage(new  
            JarvisError("stt_unsupported")),  
        true);  
else if  
    (!getProviderSettings(settings).apiKey)  
    { ui.setState("idle", "Welcome!  
    First, add your free API key in  
    Settings."); openSettings(); }  
else ui.setState("idle");
```